

Align Your Steps (AYS):

Reproduction and Extension with Reinforcement Learning

Group 14: Liu Wenzhe, Lyu Sijian, Sakpiboonchit Siratish, Wong Li Sum, Xu Luoqi, Yang Leyuan

1. Background

Our project focuses on advancing diffusion models (DMs), a class of generative models that have gained significant attention for their ability to produce highly realistic and high-quality images. These models work by gradually transforming random noise into coherent visual outputs through a series of iterative refinements, making them powerful tools for image generation. A significant challenge with DMs is their slow sampling speed, as they require numerous sequential evaluations through large neural networks, making them impractical for real-time applications.

While recent research has made strides in improving computational efficiency with advanced solvers, the optimization of sampling schedules—responsible for controlling noise levels throughout the sampling process—has largely remained underexplored. This is a missed opportunity, as fine-tuning these schedules can significantly enhance performance without adding to the computational load.

To address this gap, we propose extending the Align Your Steps (AYS) framework (Sabour et al., 2024), which customizes sampling schedules using stochastic calculus to enhance output quality. AYS leverages methods from stochastic calculus to find optimal schedules specific to different solvers, trained DMs, and datasets, outperforming previous hand-crafted schedules in various benchmarks.

Our goal is to integrate Reinforcement Learning (RL) with text embeddings, enabling dynamic adjustments to the sampling process based on input text complexity. Recent studies have demonstrated the effectiveness of RL in fine-tuning diffusion models. For instance, the DPOK framework utilizes online RL to fine-tune text-to-image models (Fan et al., 2023), enhancing image-text alignment and quality.

The expected impacts of our project are to enhance flexibility and superior quality in diffusion models and bridge the gap between sampling efficiency and output quality, making DMs more practical for real-time applications.

2. Reproduction

2.1 Algorithm Used

DMs are generative models that operate through two processes: a forward process, which gradually corrupts the image data distribution to a noise distribution, and a reverse process, which transforms the noise distribution back into image by removing noise. This transformation is mathematically formulated by Stochastic Differential Equations (SDEs), which describe how data evolved along with continuous timestamps (Song et al., 2021).

In practice, directly computing the continuous process is infeasible, so the reverse process relies on the sampling schedule to approximate the continuous transformation. The AYS algorithm proposed in this paper focuses on minimizing the discrepancy between the continuous-time SDEs and its discretized counterpart by optimizing the sampling schedule.

To find an optimal discrete schedule $(t_0, t_1, \dots, t_n)$, the authors employ the Kullback-Leibler (KL) divergence to measure the discrepancy between the true continuous-time trajectory and its discretized approximation. The KL divergence quantifies the information loss when approximating the reverse diffusion process, but directly minimizing it between complex distributions is computationally expensive. To address this, the authors introduce the KL Upper Bound (KLUB), which provides a tractable surrogate by bounding the true KL divergence. The KLUB is given by:

$$\text{KLUB}(t_0, t_1, \dots, t_n) = \sum_{i=1}^n \int_{t_{i-1}}^{t_i} \frac{s(t)^2 \dot{\sigma}(t)}{\sigma(t)^3} \mathbb{E}_{x_t \sim p_t^\theta, x_{t_i} \sim p_{t_i|t}^\theta} \left\| D_\theta \left(\frac{x_t}{s(t)}, \sigma(t) \right) - D_\theta \left(\frac{x_{t_i}}{s(t_i)}, \sigma(t_i) \right) \right\|^2 dt$$

And a tractable expression proportional to KULB metric has been derived as:

$$\text{KLUB} \propto \sum_{i=1}^n \int_{t_{i-1}}^{t_i} \frac{s(t)^2 \dot{\sigma}(t)}{\sigma(t)^3} \left(\frac{1}{\sigma(t)^2 + c^2} - \frac{1}{\sigma(t_i)^2 + c^2} \right) dt$$

To estimate the KLUB, Monte Carlo estimation is employed. But as the standard Monte Carlo method can suffer from high variance, the importance sampling is utilized, which samples from a distribution that emphasizes the important integrals that have significant impact on KLUB.

2.2. Experiment

Our experiments investigate optimal sampling schedules for DMs, addressing the inherent bias where larger timesteps produce smaller KLUB values. We developed a candidate selection strategy using asymmetric neighborhood exploration to counterbalance this effect.

2.2.1. Basic Settings

For our experimental evaluation, we selected the CIFAR10 dataset, which consists of 60,000 32×32 color images across 10 classes. We normalized the data with mean and standard deviation values of 0.5 for each in RGB channels. This normalization technique centers the data distribution and standardizes the range, to make the data distribution as we expected.

In this work, we employed the Denoising Diffusion Probabilistic Model (DDPM, Ho et al., 2020) framework for our image generation tasks. Specifically, we utilized the DDPM model that has been pre-trained on the CIFAR10 dataset as our denoiser.

2.2.2. Implementation Strategy

As mentioned, optimizing the sampling schedule is equivalent to minimizing the KLUB value, which can be computed by the product of a diffusion distance term and a reweighting factor.

However, the reweighting factor is inversely related to the timestep values. Our empirical analysis reveals this factor's behavior: as the timestep t increases, the average reweighting factor decreases significantly. This means that larger t values naturally produce smaller KLUB values, inherently biasing the optimization process toward selecting higher timesteps.

Table 1¹. Relationship between timestep values and average reweighting factors

Timestep↑	0.16	0.18	0.20	0.22	0.24
Avg. Reweighting Factor↓	4.17	4.03	3.54	2.87	2.35

Based on this analysis, we redesigned our candidate selection strategy to counteract this inherent bias. The key improvement lies in how we define the neighborhood around each timestep for potential candidates. Our implementation creates an asymmetric neighborhood where 75% of the candidate range extends toward the smaller timestep and only 25% toward the larger one.

2.3. Evaluation and Limitations

To evaluate the effectiveness of the reproduced AYS sampling schedule, we conducted experiments on the CIFAR-10 dataset using a pretrained DDPM with Denoising Diffusion Implicit Models (DDIM) sampling. The goal was to assess whether the KLUB-optimized schedule improves generation quality and efficiency relative to the baseline time-uniform schedule.

¹ Results under $(t_{\min}, t_{\max}) = (0.1, 0.3)$, average under the importance sampling strategy.

Evaluation was based on four metrics: Fréchet Inception Distance (FID), Inception Score (IS), CLIP score, and Number of Function Evaluations (NFE), which measure image fidelity, diversity, semantic alignment, and computational cost, respectively.

As shown in Table 2, the reproduced AYS schedule outperforms the baseline across all metrics. It achieves FID of 6.47, IS of 9.10, CLIP score of 0.312, and requires only 30–50 sampling steps, compared to 8.25, 8.73, 0.265, and 250 steps for the time-uniform schedule. These results indicate that the optimized schedule improves both output quality and sampling efficiency.

Table 2. Results under fixed schedule length of 11 timesteps. CLIP score computed with default prompts and random seeds. All metrics averaged over 100 CIFAR-10 samples.

Schedule	FID↓	IS↑	CLIP↑	NFE↓
Time Uniform	8.25	8.73	0.265	250
AYS (Reproduced)	6.47	9.10	0.312	30-50

Nevertheless, several limitations remain. First, the KLUB optimization process is computationally expensive, requiring over 160 iterations and approximately 6–7 hours of GPU time. Second, the evaluation was restricted to the CIFAR-10 dataset. While suitable for baseline validation, further experiments are needed to assess generalization to higher-resolution datasets such as FFHQ or ImageNet. Moreover, the use of a fixed 11-step schedule, although effective, may limit achievable performance compared to finer-grained alternatives. Lastly, the optimized schedule is highly dependent on the model architecture and sampling method. Its transferability to other configurations may be limited without retraining. These limitations motivate the development of adaptive scheduling methods. In the next section, we introduce a reinforcement learning-based approach to dynamically tailor sampling schedules according to prompt complexity.

3. Extension with Reinforcement Learning

3.1. Reinforcement Learning Framework

AYS is originally implemented as a fixed sampling schedule, optimized separately for each model and solver configuration. To enhance the flexibility and adaptability of the AYS framework, we propose integrating a Reinforcement Learning (RL) framework that dynamically adjusts the sampling schedule during image generation, based on text embeddings. The objective is to improve the alignment of generated images with input text prompts while preserving image quality and maintaining computational efficiency.

The state of the environment is defined as the text embedding $E \in \mathbb{R}^{512}$, generated using OpenAI CLIP (ViT-B/32) (Radford et al., 2021), combined with the sampling schedule $T = [t_0, t_1, \dots, t_n]$. Initially, the schedule is set to the baseline AYS schedule and is then modified by the RL agent through its actions.

Each action in this setting modifies exactly one time step of the schedule at a time. An action consists of three elements: a time step index i , which selects a position in the schedule to adjust (excluding the first and last positions that must remain fixed); an adjustment rate δt , chosen from a discrete set; and a direction of change, which determines whether the selected time step should be increased or decreased. To ensure that the updated schedule remains monotonic, the time step adjustment follows the rule:

$$t'_i = \begin{cases} t_i + \delta t(t_{i+1} - t_i), & \text{increase } t_i \\ t_i - \delta t(t_i - t_{i-1}), & \text{decrease } t_i \end{cases}$$

The total number of possible actions is determined by the number of adjustable time steps, multiplied by the number of adjustment rates, and multiplied again by two to account for both increasing and decreasing directions. Since the action space is discrete and relatively small, we employ a Deep Q-Network (DQN) (Mnih et al., 2015) to model the agent.

The reward function is designed as a linear combination of three components. First, the FID score is computed between real images and newly generated images, where we use the 10 most recent samples for FID estimation to ensure more stable statistics. Second, the CLIP score is computed between the input text prompt and the newly generated image, with both the FID and CLIP scores normalized by their respective baseline scores obtained from the initial schedule. Third, a penalty is applied based on the number of steps taken n if it exceeds a predefined maximum threshold n_{max} , encouraging the agent to find efficient adjustment sequences. The overall reward function is given by:

$$R = \lambda_{CLIP} \left(\frac{CLIP}{CLIP_{bl}} \right) - \lambda_{FID} \left(\frac{FID}{FID_{bl}} \right) - \lambda_n \max\{0, (n - n_{max})\}$$

where λ_{CLIP} , λ_{FID} , and λ_n are weight coefficients that balance the different reward factors.

The proposed algorithm is summarized in Algorithm 1.

Algorithm 1 AYS-RL

- 1: **Input:** Base schedule T_0 , text-to-image model, text embeddings, real images, prompts, number of actions per sample N
 - 2: Initialize DQN model with MultiInputPolicy; generate baseline images using prompts and T_0
 - 3: **for** each sample (text embedding E , real img I_r , base img I_0 , prompt P) **do**
 - 4: Reset state $s_0 = (E, T_0)$
 - 5: Compute baseline FID and CLIP scores from I_0
 - 6: **for** $n \in 0, \dots, N$ **do**
 - 7: Determine next action a_n
 - 8: Apply action a_n to adjust T_{n+1}
 - 9: Generate new image I_{n+1} using P and T_{n+1}
 - 10: Compute FID and CLIP scores from I_{n+1}
 - 11: Receive reward $r_n = R(FID, CLIP, n)$
 - 12: Generate new state $s_{n+1} = (E, T_{n+1})$
 - 13: Update DQN Model with (s_n, s_{n+1}, a_n, r_n)
 - 14: **Return** trained DQN model parameters
-

For our experiments, we train the RL agent using 2000 prompts and images from the COCO dataset (Lin et al., 2014). The text-to-image model used is Stable Diffusion 1.5 (Rombach et al., 2022), and we set the maximum number of actions per sample to $N = 20$.

3.2. Results and Evaluation

To evaluate the effectiveness of the AYS-RL, we compared it with the original AYS and heuristic baselines such as time-uniform and EDM schedules. To further examine AYS-RL's capability, we assessed its performance across different numbers of reinforcement learning iterations.

Table 3. Evaluation of different sampling schedules on COCO and LN-COCO samples.

Schedule	COCO				LN-COCO			
	FID↓	IS↑	CLIP↑	Inference time↓	FID↓	IS↑	CLIP↑	Inference time↓
Time-uniform	164.48	5.49	0.3161	0.33	161.13	5.85	0.3111	0.49
EDM	162.53	5.54	0.3193	1.41	160.02	5.74	0.3095	0.33
AYS	177.97	5.38	0.3153	0.51	177.07	5.62	0.3076	1.4
AYS-RL(1 iter.)	180.14	5.5	0.317	9.58	157.47	5.51	0.3094	10.26
AYS-RL(3 iter.)	173.5	5.56	0.3174	19.39	156.57	5.29	0.3085	21.26
AYS-RL(5 iter.)	171.88	5.56	0.3162	31.02	174.86	5.26	0.3087	31.89
AYS-RL(10 iter.)	186.43	5.41	0.3123	58.09	165.98	5.39	0.3073	58.35

We used the same text-to-image model, Stable Diffusion 1.5, over 10 denoising steps with each schedule on 100 randomly selected prompts and images from the COCO dataset, which were excluded from training. The results are summarized in Table 3. Across COCO samples, all schedules yielded comparable IS and CLIP scores. However, AYS-based schedules generally showed higher FID scores, likely due to FID's sensitivity to small sample sizes. This suggests heuristic schedules may perform better on small distributions. Nonetheless, Figure 1 shows that AYS-RL with only 1 iteration can produce images that are more realistic and better aligned with prompts.

Figure 1. Example outputs from different sampling schedules. Prompt: "A bald man with glasses stares forward while wearing a robot tie."



As expected with RL-based algorithms, inference time - time taken to generate an output - increases with more iterations (Dulac-Arnold et al., 2021). However, image quality and alignment do not always improve with increased iterations. AYS-RL with 3 and 5 iterations outperforms the original AYS, while the performance declines at 10 iterations. AYS-RL with 3 iterations offers the best trade-off between image quality and inference time, outperforming original AYS in both FID and CLIP while remaining substantially faster than higher-iteration variants. Figure 2 shows subtle but consistent quality improvement at 3 and 5 iterations.

Figure 2. AYS-RL outputs across different number of iterations. Prompt: "A woman leaning against a store front with an umbrella. "



To assess performance with more detailed prompts, we used the LN-COCO dataset (Pont-Tuset et al., 2020), which provides detailed and spatially grounded descriptions. Inference time remained similar across schedules. CLIP scores were slightly lower across all schedules, likely due to increased prompt complexity. However, FID improved for all schedules, especially for AYS-RL with 1 or 3 iterations. This suggests that detailed prompts help AYS-RL to generate images more closely aligned with the source image.

Figure 3 shows that generated images may align semantically with short prompts but differ from source image. When given detailed prompts, Figure 4 shows that AYS-RL captures more nuanced spatial and contextual information, which can produce outputs closer to the source image (e.g., broader scene elements or capturing background). Therefore, the choice on short or detailed prompt depends on the user intent, either to encourage diversity or closely matching the expectation (i.e, source image). AYS-RL works well with both short and detailed prompts but excels with detailed prompts compared to the other schedules. However, an excessively long prompt can lead to loss of focus and degraded quality.

Figure 3. Outputs with short prompts on different sampling schedules. Prompt: "A smiling skier poses in the snow before making her run."



Figure 4. Outputs with detailed prompts on different sampling schedules. Prompt: "in the picture we can see a woman standing on the ski boards, on the snow surface and behind her we can see the trees. "



4. Conclusions and Future Work

AYS is a general and principled framework for optimizing diffusion model sampling schedules to improve output quality. In this project, we reproduced the AYS algorithm, with modifications to the candidates control in the Monte Carlo estimation used for KLUB optimization.

However, AYS uses a fixed schedule regardless of prompt complexity. To address this, we proposed AYS-RL, which integrates reinforcement learning with text embeddings to generate adaptive schedules based on prompts. AYS-RL demonstrates improved realism and prompt alignment compared to both the original AYS and heuristic baselines, especially when detailed prompts are provided.

Future work includes training AYS-RL with more high-quality, detailed prompts to further enhance performance and evaluating its generalizability across different diffusion models and datasets. Reducing inference time remains a challenge, and future research may explore techniques such as step-reduction strategies and multiplications inference (Ivanov et al., 2023) to improve efficiency.

5. References

- [1] Dulac-Arnold, G., Levine, N., Mankowitz, D. J., Li, J., Hester, T., Barth-Maron, G., Bellemare, M. G., and Graf, J. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning*, 110, 2419–2468, 2021.
- [2] Fan, Y., Watkins, O., Du, Y., Liu, H., Ryu, M., Boutilier, C., Abbeel, P., Ghavamzadeh, M., Lee, K., & Lee, K. DPOK: Reinforcement Learning for Fine-tuning Text-to-Image Diffusion Models. 2023.
- [3] Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33, 6840–6851, 2020.
- [4] Ivanov, D. A., Larionov, D. A., Kiselev, M. V., Vanyukov, P. M., and Polezhaev, I. V. Deep reinforcement learning with significant multiplications inference. *Scientific Reports*, 13, 20865, 2023.
- [5] Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. Microsoft coco: Common objects in context. In *Computer Vision–ECCV 2014: 13th European*

Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part V 13, pp. 740–755. Springer, 2014.

[6] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[7] Pont-Tuset, J., Uijlings, J., Changpinyo, S., Soricut, R., and Ferrari, V. Connecting vision and language with localized narratives. *European Conference on Computer Vision (ECCV)*, 2020.

[8] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pp. 8748–8763. PmLR, 2021.

[9] Rombach, R., Blattmann, A., Lorenz, D., Esser, P., and Ommer, B. High-resolution image synthesis with latent diffusion models. *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 10674–10685, 2022.

[10] Sabour, A., Fidler, S., & Kreis, K. Align Your Steps: Optimizing Sampling Schedules in Diffusion Models. 2024.

[11] Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. Score-Based Generative Modeling through Stochastic Differential Equations. *ICLR*, 2021.